Deep Network Development

THEORY TEST 1 • 360° SKIM REFERENCE

Autumn 2026 | ELTE | Lectures 1–6 + examinable NumPy practice weeks 3–6

How to use this reference

Read the **first-page decision rules** and **true/false traps** immediately before the test. Use the remaining sections for exact formulas, model dimensions, worked NumPy results and memory-sharing behavior. Example computations were checked with NumPy; the formulas were cross-referenced with the lecture slides.

Official assessment format

Each theory question usually gives **3–6 statements** with **at least one true**. Full points require selecting **all and only** the true statements. Exactly **one selection error** receives half points; both selecting a false choice and omitting a true choice are separate errors.

The official example-question list says it is **not exhaustive**. It specifically names **Lectures 1–6** and **NumPy in the practice videos for weeks 3–6**; **ndarray.strides** and **stride tricks** are excluded.

Accuracy policy

This is an independently assembled study aid, not a replacement for any later official announcement. Statements are sourced to the uploaded slides/example list. When general mathematical conventions and slide conventions differ, the distinction is written explicitly. No guide can honestly guarantee the exam will contain nothing beyond the supplied examples.

Legend for source labels

TT1 = theory_test1_examples_26autumn.md. **L1 … L6** = dndmsc26autumn_prez1_en.pdf … prez6_en.pdf. **P3 … P6** = dnd26autumn_practice_week3...week6_video_prez.pdf. **N1 … N7** = supplemental practice-note PDFs. A page reference denotes the PDF page, not an extracted-text line.

01 | The 90-second recall sheet

Sources: TT1 entire example-question list; L1–L6; P3–P6.

Remember	Remember
Supervised: labeled (x, y). Regression = numerical target; classification = class.	**Unsupervised:** unlabeled structure (clusters, reductions, denoising). RL uses reward signals.
Hypothesis: x and learned θ → predicted ŷ. Training uses (x, y) and scalar loss J.	**Linear:** $\hat{y} = X\theta$; n features → n+1 parameters with intercept.
Regression: identity output, squared-error loss, unconstrained real-valued ŷ.	**Binary classification:** σ(Xθ), output $0 < p < 1$, BCE/logistic loss.
Multiclass: k logits → softmax probabilities (sum 1), cross-entropy.	**Full-batch GD:** gradient over all m; mini-batch SGD: subset per update.
Convex linear least squares: descent can reach global minimum; MLP not guaranteed.	**Overfitting:** train good, validation bad. Stop using **validation**, never test.
Feature scaling: min-max (x−min)/(max−min); z-score (x−mean)/std.	**Neural parameters:** each dense layer has d_out × (d_in+1) scalars.
NumPy shapes: shape tuple, ndim int, size int, dtype dtype object.	**Indexing:** slice typically view; integer-list/mask selection copy.
Broadcast: align right; each size equal or one = 1; otherwise error.	**Reduction:** axis=0 removes rows, axis=1 removes columns (unless keepdims).
np.array.T: transpose swaps axes; for 1D, shape stays the same.	**Backprop:** $c=b^2 \Rightarrow \partial J/\partial b=(\partial J/\partial c)\cdot 2b$; $c=a+b \Rightarrow$ same gradient to each input.

Two prototype dimensions to memorize

200 cases, 4 features (regression): X (200,5), θ (5,), y and ŷ (200,), **5 parameters**.
100 cases, 2 features (binary): X (100,3), θ (3,), y and ŷ (100,), **3 parameters**.
MLP 20→10→1: first layer 210, second layer 11, total **221**.

02 | What exactly is being optimized?

Sources: L1 pp. 25–40, 56–77; L2 pp. 7–21; TT1, first question group.

Learning regimes and tasks

Setting	Given	Goal / standard examples
Supervised	Features x together with correct labels y.	Regression predicts a continuous/numeric quantity; classification predicts one of discrete categories.
Unsupervised	Inputs x without target y.	Clustering groups related examples; dimensionality reduction compresses representations; denoising recovers cleaner input structure (no externally supplied class labels).
Reinforcement	An agent acts in an environment and receives **rewards** (often delayed).	Learn a policy to maximize cumulative reward; the environment generally does **not** provide the correct action label at every step.

Single example, full training sample, and objective

- A supervised **training example** is $(x^{(j)}, y^{(j)})$: x has input feature(s), y is the target. Different examples use index j; different features use index i.
- The **hypothesis** $h_\theta(x)$ outputs an estimate ŷ. Parameters θ are chosen during training; they are not input features. At prediction time, y is unknown.
- For m examples, X stores all feature rows; y stores labels; ŷ stores predictions. Label y can be a **scalar** for one output or a **vector** for multiple outputs; a batch of vectors forms a matrix.
- The **loss/objective** J(θ) measures prediction disagreement and returns **one scalar** when aggregated for optimization, even if individual predictions/labels are vectors.

Vocabulary you must distinguish

Parameter = fitted value (weight/bias in θ or Θ). **Hyperparameter** = configuration chosen using validation data (learning rate, degree, width/depth, stopping epoch, often regularization strength). **Feature** = input x or column in X. **Label** = true target y. **Loss** = scalar to minimize, **not** x, y, θ itself, or the activation function.

Geometric intuition

- One real feature in linear regression: hypothesis is a **line** in an x–y plot. Two features: a **plane** in x_1–x_2–y. n features: a hyperplane in n+1-dimensional space.
- A logistic regression classifier also uses a **linear decision boundary in feature space**, since the p=0.5 boundary is $\theta^T x = 0$ (using intercept convention).

03 | Linear regression and shapes

Sources: L2 pp. 16–21; L3 pp. 20–34; L4 pp. 6–23; TT1 cholesterol example.

The model

Definitions

A one-output linear model: $\hat{y} = \theta_0 + \theta_1 x_1 + \ldots + \theta_n x_n$. Introduce dummy feature $x_0 = 1$, then $\hat{y} = x^T\theta$ for one point and $\hat{y} = X\theta$ for a full dataset. With n actual input features, there are **n+1 trainable scalar parameters**, including θ_0 (intercept).

- For **m** training cases, **n** features, and one output: **X of shape (m,n+1)** (the first column is 1), θ shape **(n+1,)**, y and ŷ shape **(m,)**.
- Raw features (age, weight, sex encoding, etc.) are stored in **X**. Known target values (cholesterol) belong in **y**. θ contains learned coefficients, not the raw features.
- Linear regression has **unbounded real-valued predictions**. The task might have physically meaningful positive values, but the plain mathematical model does not enforce positivity.

Exact example: 200 patients, four features

Object	Meaning	Shape / count
X	200 rows, 4 features + 1 intercept column	**(200, 5)**
θ	θ_0 intercept and four slopes	**(5,)** = 5 scalars
y	Observed cholesterol levels	**(200,)**
ŷ = X @ θ	One predicted cholesterol per patient	**(200,)**
J(θ)	Dataset-aggregated loss	**Scalar**

Least-squares objective and gradient

Course convention (the “half MSE” objective)

$\mathbf{J(\theta) = (1 / 2m) \sum_{j=1}^{m} (\hat{y}^{(j)} - y^{(j)})^2 = (1 / 2m) \|X\theta - y\|^2}$.
$\mathbf{\nabla J(\theta) = (1/m) X^T(X\theta - y)}$.
$\mathbf{\theta \leftarrow \theta - \alpha \nabla J(\theta)}$ (update all coefficients simultaneously using gradients from the same current θ).

- The 1/2 cancels a factor of 2 during differentiation. Some libraries define MSE as (1/m)·Σerror² without the 1/2; the minimizer is unchanged, but gradients/loss values differ by a factor of 2.
- A gradient is the direction of **steepest local increase**; taking the **negative gradient** decreases loss locally for a sufficiently small step α.
- For least squares, the quadratic J is **convex**, so appropriately run gradient descent can approach a **global** minimizer. Uniqueness requires full column rank; rank-deficient designs may have many minimizers.

04 | Normal equation, regularization, scaling

Sources: L3 pp. 35–43; L4 pp. 17–23, 70–79; L5 pp. 4, 9; TT1 normal-equation questions.

Closed-form least squares

Normal equation and dimensions

$\theta^* = (X^TX)^{-1} X^Ty$, provided **$X^TX$ is invertible**.
X: (m, n+1); X^TX: (n+1, n+1); $(X^TX)^{-1}$: (n+1,n+1); X^Ty: (n+1,); final θ^*: (n+1,).
If singular or for general stable least squares, use **$\theta^* = X^+y$** (Moore–Penrose pseudoinverse; e.g., **np.linalg.pinv(X) @ y**).

- The slides show the **regularized normal equation** as **$\theta = (X^TX + \lambda I)^{-1} X^Ty$**. I is the **identity matrix of shape (n+1,n+1)**, not data, a label, or a learned parameter. $\lambda \geq 0$ controls penalty strength.
- **Important convention:** the later slide illustrating an L2 loss penalizes $\Sigma_{i=1}^{n} \theta_i^2$ (excluding the bias θ_0). If the intercept is unpenalized, the closed form uses a penalty matrix with a **0 in the intercept position** rather than a full identity. The scale of λ also depends on the objective's factors. Do not assume all printed λ formulas are numerically identical without checking the definition.

Why feature scaling matters

- Different feature magnitudes yield elongated loss contours; gradient descent can zigzag and converge slowly. Scale **each numerical feature independently**. The intercept column of 1 is not a feature to standardize.

Method	Formula for a feature x	Properties
Min–max	$x' = (x - \min_{train}) / (\max_{train} - \min_{train})$	Training values lie in [0,1] when max≠min; extremely outlier-sensitive. New data can fall outside [0,1].
Standardization / z-score	$x' = (x - \mu_{train}) / \sigma_{train}$	Training mean 0 and standard deviation 1 when $\sigma>0$. Lecture says this is **less sensitive to outliers** than min–max, but it is not outlier-proof.

- Fit scaling statistics **only on training data**; apply the same values to validation and test data. For a constant-valued feature, the denominator is zero: remove the feature or explicitly handle the division.

Over/underfit remedies linked to complexity

- **High bias / underfit:** both training and validation losses high. Increase model capacity, improve features, or train appropriately. **High variance / overfit:** training loss low yet validation loss high. More training data, simpler model, L2 regularization, or early stopping can help.

05 | Logistic regression and binary classification

Sources: L3 pp. 44–97; L4 pp. 25–29; TT1 two-feature illness example.

Model, interpretation and ranges

Binary logistic regression

$\mathbf{z = \theta_0 + \theta_1 x_1 + \ldots + \theta_n x_n = X\theta}$
$\mathbf{\sigma(z) = 1 / (1 + e^{-z})}$
$\mathbf{\hat{y} = p(y=1|x) = \sigma(z)}$; for every **finite real z**, $\mathbf{0 < \sigma(z) < 1}$. Use 0.5 threshold by default: $p<0.5 \rightarrow$ class 0; $p \geq 0.5 \rightarrow$ class 1. sigmoid(0)=0.5.

- Despite the word “regression”, **logistic regression is a supervised binary classification method**; ordinary linear regression is a supervised numerical regression method.
- The sigmoid produces probabilities, not discrete labels; thresholding probabilities produces labels. A logistic decision boundary is linear in the input features for a linear logit model.

Binary cross-entropy / logistic loss

Use BCE, not least squares, for the standard logistic model

$\mathbf{J(\theta) = -(1/m) \Sigma_j [\, y^{(j)} \log p^{(j)} + (1-y^{(j)}) \log(1-p^{(j)}) \,]}$, where $p^{(j)}=\sigma(x^{(j)T}\theta)$.
For y=1: loss = $-\log(p)$. For y=0: loss = $-\log(1-p)$. Minimize **J(θ)**, not x, y, g, or the magnitude of θ alone.
$\mathbf{\nabla J(\theta) = (1/m) X^T(p-y)}$ for this model.

- For the standard logistic model, binary cross-entropy is **convex in θ**, whereas squared loss after the sigmoid need not be convex. This convexity does **not** apply to an arbitrary MLP. Perfectly separable, unregularized datasets can have an optimum approached only as weight magnitudes grow.

Exact example: 100 patients, two blood features

Item	Answer
Task	Ill vs healthy: **binary classification** → logistic regression.
Training objects	X **(100,3)** with intercept; θ **(3,)**; y and p **(100,)**.
Parameters	**3** trainable scalars (θ_0, θ_1, θ_2).
Prediction range	Each p strictly between 0 and 1 for finite input/weights.
Raw laboratory values	In X / features x_1 and x_2; the illness state is y.
Optimized quantity	A scalar **BCE loss** J(θ).

06 | Model complexity and evaluation

Sources: L4 pp. 33–93; L5 pp. 6–13; TT1 overfitting and hyperparameter items.

Three disjoint roles for data

Split	What it does	Forbidden shortcut
Training	Fits learnable parameters θ/Θ by optimization.	Do not treat its low error as proof of generalization.
Validation	Chooses hyperparameters, model capacity, and stopping checkpoint.	Do not use final test set repeatedly to tune these choices.
Test	Reports final held-out performance after model selection.	Do not use test loss to decide early stopping or architecture.

How the learning curves should be read

Observed pattern	Likely explanation	Usual next step
Train high, validation high	Underfitting / model too constrained or not adequately trained	Increase useful model capacity / improve optimization.
Train much lower than validation	Overfitting / poor generalization	Regularize, simplify model, add training data, early-stop.
Train decreases; validation rises	Overfitting growing as training proceeds	Restore best checkpoint chosen by validation.
Validation and train low, comparable	Good empirical fit (provided both measured correctly)	Evaluate once on test. Errors may still exist.

Early stopping and hyperparameter search

- The correct early-stopping signal is **validation loss**, often with a patience period and restoration of best-validation model. **Never stop because the test loss began rising** — that leaks evaluation data into training decisions.
- Training loss often keeps decreasing even while validation loss rises; therefore “stop when training loss rises” is **not the rule**.
- Learned weights and biases are usually updated with gradient methods. Learning rate α, architecture, polynomial degree, batch size, regularization setting, and stopping epoch are **hyperparameters**, generally chosen using validation performance rather than directly updated with ordinary weight-gradient descent.
- **More training examples** often reduce overfitting; **fewer training examples** often worsen it. **More model parameters** can increase overfitting risk; **fewer parameters** may reduce overfitting but can cause underfitting. These are **tendencies, not universal guarantees**.
- **Changing the number of test examples** with fixed training procedure does **not by itself change the learned model’s tendency to overfit**; it changes precision/reliability of the final performance estimate.

07 | Neurons, layers and parameter counting

Sources: L4 pp. 97–106; L5 pp. 21–65, 81–98; L6 pp. 2–8; TT1 MLP hypothesis question.

One artificial neuron

- A neuron combines a weighted input and a bias then applies activation: **$a = g(w^T x + b)$**. With g = sigmoid it has logistic-regression-like output. A single sigmoid neuron with linear input weights gives a linear decision boundary.

Dense MLP and why nonlinearity is essential

Two-layer MLP from the official example

$h(x) = g_2[\, W_2 \cdot g_1(W_1 x + b_1) + b_2 \,]$.
For d_{in} inputs, h hidden neurons, k outputs:
W_1: (h,d_{in}), b_1: (h,); W_2: (k,h), b_2: (k,).
Parameter count = $h(d_{in}+1) + k(h+1)$.

- **Hidden activation g_1 must be nonlinear** to gain expressive power. If all intermediate layers are linear/identity, the whole composition is still an affine function; a deep linear network cannot learn curved or XOR-like decision boundaries simply by stacking linear layers.
- The **output activation g_2 depends on the task**; it can be omitted/identity in regression, sigmoid for binary probability, softmax for mutually exclusive multiclass classification.

20 inputs → 10 hidden → 1 numerical output

Layer	Weights	Biases	Scalars
First dense	10 × 20 = 200	10	**210**
Second dense	1 × 10 = 10	1	**11**
Whole network			**221**

- For a single continuous target: keep **g_1 nonlinear**, choose **g_2 identity** (or omit it), and use MSE. **Sigmoid output is generally inappropriate for unconstrained regression** because it restricts the prediction to (0,1).

Task → output layer → standard loss

Task	Output(s)	Activation	Loss
Numerical regression	k real numbers	Identity (none)	MSE, averaged over target elements
Binary classification	1 probability	Sigmoid	Binary cross-entropy
Mutually exclusive k-class	k probabilities, sum 1	Softmax	Categorical cross-entropy

- **Multi-label vs multiclass:** mutually exclusive k classes use softmax; k independent yes/no labels usually use k sigmoids and binary cross-entropy. The uploaded lecture explicitly develops softmax for multiclass classification.

08 | Activation functions: recognize by shape

Sources: L5 pp. 56–64, 90–98; L6 p. 8; TT1 activation graph questions.

Function	Expression	Output range	Recognizable graph / property
Identity	$g(z)=z$	All real numbers	Straight diagonal line; appropriate unrestricted regression output.
Sigmoid	$\sigma(z)=1/(1+e^{-z})$	Strictly (0,1) for finite z	Smooth S-curve, centered at (0,0.5); $\sigma'=\sigma(1-\sigma)$.
tanh	$\tanh(z)$	Strictly $(-1,1)$ for finite z	S-shaped, odd, centered at (0,0); derivative $1-\tanh^2(z)$.
ReLU	$\max(0,z)$	$[0,\infty)$	Flat at 0 for $z<0$; slope 1 for $z>0$; nondifferentiable exactly at 0.
Softmax	$e^{z_i} / \Sigma_j e^{z_j}$	Per coordinate (0,1); sum=1	Vector-to-vector normalization. Each class gets a probability.

- Neural network output can be **scalar or vector**. For multi-output regression, the last layer has one neuron per target. For 10 exclusive classes such as digits 0–9, it has **10 outputs**, and probabilities must sum to one.
- One-hot target y for class index c has **y[c]=1** and all other entries 0. Multiclass cross-entropy for one case is $\mathbf{-\Sigma_i\, y_i \log(p_i) = -\log(p_{true\ class})}$.
- Softmax is not just “apply sigmoid separately”. Independent sigmoids need not sum to 1; softmax couples the outputs.

Implementation distinction (supplemental, not a new exam assumption)

In PyTorch, **nn.CrossEntropyLoss** normally accepts raw **logits** (it incorporates log-softmax internally). **nn.BCEWithLogitsLoss** takes a raw binary logit and incorporates sigmoid stably. The lecture’s conceptual final activations (sigmoid/softmax) still describe predicted probabilities.

09 | Gradient descent, mini-batch SGD, backprop

Sources: L5 pp. 65–72; L6 pp. 9–11, 16–76; TT1 SGD and computational graph questions.

Batch size trade-offs

Term / setting	Meaning and impact
Full-batch gradient descent	Uses **all m** training examples to estimate the gradient at each update. Computationally costly for large m; gradient of training objective is exact.
Mini-batch SGD	Samples **b** examples per update, where typically 1<b<m; less costly than full batch and has gradient noise.
Strict “online” SGD	Uses b=1 per update. In the lecture, “SGD” is used broadly for random mini-batches; therefore “SGD must use one single example” is **false** in course terminology.
Increase batch size	More memory and computation per update; gradients usually less noisy and more representative; typically fewer updates per epoch.
Decrease batch size	Less memory per update; noisier gradient direction; generally more updates per epoch. This is not automatically faster or more accurate.

Computational graphs and the chain rule

- A graph records elementary operations and dependencies. During **forward pass**, compute intermediate values and loss; in **backward pass**, pass gradients backward through local derivatives. Contributions from multiple downstream paths **add**.

Forward node	Given upstream grad $g_c = \partial J/\partial c$	Backward contributions
$c = b^2$	$\partial c/\partial b = 2b$	$\mathbf{g_b \mathrel{+}= g_c \cdot 2b}$
$c = a+b$	$\partial c/\partial a=1$; $\partial c/\partial b=1$	$\mathbf{g_a \mathrel{+}= g_c}$; $\mathbf{g_b \mathrel{+}= g_c}$
$c = ab$	$\partial c/\partial a=b$; $\partial c/\partial b=a$	$\mathbf{g_a \mathrel{+}= g_c \cdot b}$; $\mathbf{g_b \mathrel{+}= g_c \cdot a}$
$c = \sigma(b)$	$\partial c/\partial b=c(1-c)$	$\mathbf{g_b \mathrel{+}= g_c \cdot c(1-c)}$

- “+=” emphasizes that the same variable may affect J through several paths; at a node with one downstream use, it reduces to “=”. For **c=b²**, the correct factor is **2b**, not 2c.

Training loop conceptual sequence

- **1** Initialize parameters → **2** choose a training mini-batch → **3** forward predictions and loss → **4** backpropagate gradients → **5** update parameters → repeat. Evaluate validation loss separately for early stopping. The lecture also describes symbolic/local derivative derivation as a preparatory step.

10 | NumPy foundations and indexing

Sources: P3 pp. 45–89; P4 pp. 15–33; P5 pp. 4–16; N3 pp. 3–12; TT1 NumPy questions.

The four core attributes

Attribute	Type and precise meaning	Example for a=np.zeros((32,5))
a.shape	**tuple** of axis lengths; can contain zero and ones	(32, 5)
a.ndim	**int** number of axes = len(shape)	2
a.size	**int** total elements = product(shape)	160
a.dtype	NumPy **dtype** describing element type	dtype('float64')
a.itemsize	**int** bytes per element	8 on float64

- Valid shapes include **(1,)** and **(0,1,0)**. A shape is not a scalar integer such as 12. Zero-length axes are valid; size then equals 0. A NumPy array has homogeneous dtype except object references.

Indexing rules (as they are tested)

- A single integer index **removes** that axis. A slice, including a length-1 slice, **preserves** the axis. Omitted trailing axes behave as ":". **Ellipsis ...** fills missing full slices.

Expression for a.shape=(32,5)	Output shape	Reason
a[:, :]	**(32,5)**	Two slices.
a[:, 1]	**(32,)**	Column indexed; axis 1 disappears.
a[1, :]	**(5,)**	Row indexed; axis 0 disappears.
a[1:, :]	**(31,5)**	31 rows retained.
a[0, :-1]	**(4,)**	Last column excluded.
a[[0], :]	**(1,5)**	List indexing preserves length-1 selection.
a[[-1, -1], -1]	**(2,)**	Last row chosen twice; last column scalar.

Basic vs advanced

- **Basic:** integers, slices, ellipsis and None/newaxis. A non-scalar basic slice normally returns a **view** sharing data. **Advanced:** integer arrays/lists or Boolean masks return a **copy** when read.
- **Important exception in wording:** writing **a[[1,2]]=99** writes into the **original** array, but first assigning **b=a[[1,2]]** and then b[:]=99 changes only b. Similar for Boolean masks.

11 | NumPy broadcasting, axes and dimensions

Sources: P4 pp. 47–65; P5 pp. 32–84; N4 pp. 2–9; TT1 explicit shape cases.

Broadcasting algorithm: compare right to left

- Pad shorter shapes with leading 1s. At each aligned axis, dimensions must be **equal** or **one of them must equal 1**. The output axis takes the larger dimension; otherwise **ValueError**. Broadcasting is not a matrix product.

Operands	Result	Why
(32,5) + (32,)	**Error**	Compare last axes: 5 versus 32, neither 1.
(32,5) + (32,1)	**(32,5)**	Per-row scalar: last-axis 1 broadcasts to 5.
(32,5) + (1,32)	**Error**	Last axes 5 and 32 conflict.
(32,5) + scalar 1	**(32,5)**	Scalar broadcasts everywhere.
(32,5) + (32,1,5)	**(32,32,5)**	(1,32,5) vs (32,1,5), all axes compatible.
(2,2) × (2,2,1)	**(2,2,2)**	Shapes: (1,2,2) × (2,2,1).

Axes, reduction, transpose and newaxis

Operation on shape (32,5)	Result	Interpretation
a<0; a+a	(32,5)	Elementwise comparison/addition.
a.T; a.T.T	(5,32); (32,5)	Transpose swaps two axes; double transpose restores shape.
np.mean(a,axis=0)	**(5,)**	Reduce along rows → one mean per column.
np.mean(a,axis=1)	**(32,)**	Reduce along columns → one mean per row.
np.sum(a,axis=0)	**(5,)**, ndarray	Non-reduced axis remains.
np.sum(a)	**()**, NumPy scalar	All axes collapse; not ndarray for ordinary numeric arrays.
a[:,None,:]	**(32,1,5)**	Add a length-one axis in the middle.
np.mean(a,axis=1,keepdims=True)	**(32,1)**	Reduced dimension stays as length 1.

- For a two-dimensional x, the matrix product **x @ w** requires inner dimensions to match. **x * w** is elementwise multiplication after broadcasting. **reshape** changes shape but preserves the number of elements.

12 | NumPy worked examples from the official list

Sources: TT1 NumPy sample prompts; corroborated by P3–P6, N3–N5. Outputs validated with NumPy.

A = np.arange(4).reshape(2, −1)

The actual values are **A = [[0,1],[2,3]]**; A.shape is **(2,2)**.

Expression	Correct result
A * 2 + 1	shape **(2,2)**; values [[1,3],[5,7]]
A[A>0]	shape **(3,)**; values [1,2,3]
A[(A>0) & (A%2==0)]	shape **(1,)**; value [2]
A * A.reshape(2,2,−1)	shape **(2,2,2)**
np.count_nonzero(A==0)	**1**
np.any(A==0)	**True**
np.all(A==0,axis=1)	**[False, False]**

V = np.arange(4) — does changing a derived array change V?

b constructed as…	Then b[:]=99: does V change?	Reason
V.reshape(2,2)	**Yes**	View, for this contiguous source.
V.T	**Yes**	For 1D, transpose returns same array data/shape.
np.append(V,42)	**No**	Returns new array.
V[1:2]	**Yes**	Slice view.
V[[1,2]]	**No**	Advanced integer-index copy.
V[V<2]	**No**	Boolean-index copy.
V+V	**No**	Elementwise result is new array.

C = np.array([3,1,2])

Expression	Actual output
np.unique(C)	[1,2,3]
np.unique(C==0)	[False]
np.bincount(C)	[0,1,1,1] (counts for 0,1,2,3)
np.sort(C)[::-1]	[3,2,1]
np.argsort(C)	[1,2,0]
C[C!=0]	[3,1,2]

13 | NumPy concepts that create MCQ traps

Sources: P3 pp. 57–89; P4 pp. 28–66; P5 pp. 17–100; P6 pp. 2–36 excluding strides; N3–N5.

Copies, views, reshaping, and assignment

- **a.reshape(...)** can create a view when layout permits, as in the official contiguous np.arange examples; it may **copy** for other memory layouts. **a.T** swaps axes without copying for a normal ndarray. **a.copy()** guarantees independent storage. **np.append** returns a new array; it does not modify a in place.
- **Read** by integer-array or boolean-mask indexing creates a copy. **Direct assignment** through such an index targets a. With repeated advanced indices, **+=** may not accumulate duplicates as expected (buffered semantics).
- A Boolean condition on an array has the same shape as the input. Filtering with a matching full-mask **a[mask]** generally creates a **flat 1D selection** containing the true elements in row-major order.

Operator meaning

- **+**, *, **, comparisons act elementwise with broadcasting. @ / **np.matmul** uses matrix product; a @ b means contracted inner dimensions. **&** and **|** combine Boolean arrays; put parentheses around both comparisons: **(a>0) & (a%2==0)**. Do not use Python **and**/**or** for elementwise array logic.

Common functions

Method	High-yield property
np.arange(start,stop,step)	Stop excluded, as in range; np.linspace(start,stop,num) includes both endpoints by default.
np.sum/mean/any/all(a,axis=k)	Remove named axis by default; **keepdims=True** retains it with size 1.
np.argmax(a,axis=k)	Returns **indices** of maximum along axis k; not max values.
np.nonzero(mask)	Tuple of coordinate arrays, one per dimension; each has one item per True entry.
np.argwhere(mask)	Matrix of matching coordinates, shape (number of matches, ndim).
np.unique(a)	Sorted unique values by default; return_counts=True adds counts.
np.bincount(a)	Counts nonnegative integer codes from 0 through max(a), including zero-count codes.
np.argsort(a)	Indices that would sort the array, not the sorted values.
np.count_nonzero(mask)	Count True entries; np.any detects any; np.all requires every entry True.

Deliberately excluded from theory test 1

The official NumPy example file explicitly states that **ndarray.strides and stride tricks are NOT included**. You still need the concept of **views sharing data**, not the byte-stride calculations.

14 | True/false exam-trap ledger (part 1)

Sources: Each statement is a direct conclusion from TT1's example topics, checked against L1–L6 and P3–P6. Interpret details literally.

Statement	Verdict	Reason
The hypothesis directly uses the true y as an input when predicting an unseen x.	FALSE	It predicts from x and learned parameters; y is the training target.
A supervised example contains both features and a target.	TRUE	(x, y).
The aggregate objective J for standard training is a scalar.	TRUE	It can sum/average many per-example or per-output losses.
Logistic regression is an unsupervised regression algorithm.	FALSE	It is supervised classification despite its name.
Linear regression $\hat{y}$ must lie between 0 and 1.	FALSE	No such output constraint in plain linear regression.
Sigmoid of a finite real logit can equal exactly 0 or 1 mathematically.	FALSE	Strict range (0,1); finite-precision numerical saturation is separate.
One feature in affine regression requires only one parameter.	FALSE	Slope plus intercept: 2.
X for 100 cases with 2 features plus intercept has shape (100,3).	TRUE	Assumes intercept column included as in the slides.
All matrices X^TX can be inverted.	FALSE	Can be singular; pseudoinverse or regularization avoids this.
The identity matrix in the ridge equation has same dimensions as X.	FALSE	It matches X^TX, shape (n+1,n+1).
Least-squares loss is convex in linear model coefficients.	TRUE	Global minimizers; uniqueness needs suitable rank.
MLP loss is generally convex in its weights.	FALSE	Nonlinear networks have nonconvex objectives.
A gradient points in the direction of steepest local decrease.	FALSE	Gradient points toward steepest increase; use negative gradient.
The standard BCE objective for linear logistic regression is convex in θ.	TRUE	Not to be confused with arbitrary MLP BCE objective.
Normalization always makes validation/test observations lie in [0,1].	FALSE	Training-fitted min-max stats do not bound new data.
Min–max scaling is less affected by outliers than z-scoring.	FALSE	Lecture explicitly states min–max is more sensitive.

15 | True/false exam-trap ledger (part 2)

Sources: TT1 sample prompts: MLPs, early stopping, training strategy and NumPy.

Statement	Verdict	Reason
Lower training loss alone proves a better model.	FALSE	Check validation and final test generalization.
Early stopping should track the test loss.	FALSE	Track validation loss; test is only for final evaluation.
Increasing test-set size by itself reduces overfitting of the same trained model.	FALSE	It improves estimation precision, not training dynamics.
Fewer training examples can make overfitting more likely.	TRUE	A common tendency, not unconditional.
The weights of an MLP are standard hyperparameters.	FALSE	They are learned parameters; widths and rates are hyperparameters.
A two-dense-layer network without hidden nonlinear activation can represent any curved boundary.	FALSE	Composition of affine maps is affine.
A 20→10→1 network with biases has 221 trainable scalars.	TRUE	210 first + 11 second.
Softmax probabilities of an exclusive k-class output sum to 1.	TRUE	Vector output with normalized exponential.
SGD always means a single sample per update in these slides.	FALSE	Slides allow a randomly chosen mini-batch.
Smaller mini-batches generally give noisier gradients.	TRUE	Higher gradient-estimate variance.
At $c=b^2$, the backward multiplier is 2c.	FALSE	It is 2b.
At c=a+b, the gradient from c flows to both a and b unchanged.	TRUE	Each local derivative is 1.
A.shape can equal the integer 12.	FALSE	shape is a tuple; size can be 12.
a[1,:] and a[1:2,:] have the same shape.	FALSE	Indexed axis collapses vs sliced axis retained.
Boolean advanced indexing gives a view over the same data.	FALSE	Reading with a boolean mask returns a copy.
Broadcasting (32,5)+(32,) succeeds.	FALSE	Trailing 5 vs 32 is incompatible.
For shape (32,5), a[:,None,:] has shape (32,1,5).	TRUE	None inserts length-one axis.
np.sum(a) always has shape (1,).	FALSE	Ordinary total reduction yields a scalar/shape ().

16 | Practice appendix (outside stated TT1 scope)

Sources: Supplement only: N1 = Python Fundamentals; N2 = Classes, Iterators and Generators; N6 = Describing a Table; N7 = Tensors, Gradients and Regression; P7 video. Official TT1 scope does not list these as theory-test content.

Python and algorithms in 10 reminders

- Python is dynamically but strongly typed; a variable is a reference to an object. Syntax errors stop compilation; type errors can appear only when the problematic line executes. Restart a notebook kernel and rerun all cells to catch hidden state.
- List indexing O(1); membership in an unsorted list O(n); append amortized O(1); repeated middle deletion $O(n^2)$ total in the worst case. Sets and dicts average O(1) membership/lookup, assuming suitable hashing.
- Immutable hashable tuples can be dictionary keys if all elements are hashable; mutable lists cannot. Comprehensions eagerly build lists/sets/dicts; generator expressions produce items lazily.
- Class instances preserve their own state via **self**; **__init__** initializes it. **__len__** supports len(), **__getitem__** supports indexing, and iterators implement **__iter__** and **__next__**, raising **StopIteration** at exhaustion.

Data-table preparation and descriptive statistics

- CSV parsing is not the same as string.split(","): quoted delimiters/newlines require csv.reader. Missing numerical data are commonly encoded as **NaN**; ordinary mean/sum can propagate NaN, while np.nanmean/np.nansum ignore NaNs.
- Summaries include mean, median, std, quantiles, group counts; histograms count observations by bin, scatter plots compare two variables, correlation measures **linear** association (neither causality nor all nonlinear association).
- Prepare numeric network input by encoding categories (often one-hot), imputing missing numeric values using training-derived statistics, optionally adding missingness indicators, scaling features on training only, and applying the same transformations to validation/test.

PyTorch: quick bridging reminders

- **torch.Tensor** is an n-D tensor that has shape, dtype, device and gradients. PyTorch float literals default to float32; converting NumPy float64 via torch.tensor preserves float64 unless specified. Neural layer input and parameters must have compatible dtype and device.
- **torch.tensor(np_array)** copies; **torch.from_numpy(np_array)** shares memory; a CPU tensor's **.numpy()** may share memory if layout/dtype and grad constraints allow. For a trainable tensor: **loss.backward()** computes gradients; reset using optimizer.zero_grad(); then optimizer.step() updates parameters.
- A standard network training pass is model.train() → forward → scalar loss → zero gradients → backward → step; validation uses model.eval() with torch.no_grad(). **nn.MSELoss** default is mean squared error (without lecture's extra 1/2 factor).

17 | Source and discrepancy notes

Sources audited in the guide

Code	Uploaded document	Primary use
TT1	theory_test1_examples_26autumn.md	Assessment format, exact theory scope, NumPy exclusion, practice questions.
L1	dndmsc26autumn_prez1_en.pdf	Core ML: supervised, unsupervised, reinforcement, task framing.
L2	dndmsc26autumn_prez2_en.pdf	Regression, supervised loss, least squares.
L3	dndmsc26autumn_prez3_en.pdf	Multivariate linear regression, scaling, classification, logistic loss.
L4	dndmsc26autumn_prez4_en.pdf	Normal equation, polynomial regression, overfit, validation, early stopping.
L5	dndmsc26autumn_prez5_en.pdf	MLPs, parameter counts, activation functions, SGD, multiclass.
L6	dndmsc26autumn_prez6_en.pdf	Chain rule and backpropagation mechanics.
P3	dnd26autumn_practice_week3_video_prez.pdf	NumPy 1D arrays, masks, creation and basic views.
P4	dnd26autumn_practice_week4_video_prez.pdf	Multidimensional arrays, axes, indexing, reductions.
P5	dnd26autumn_practice_week5_video_prez.pdf	Advanced NumPy indexing, broadcasting, reshape/transpose.
P6	dnd26autumn_practice_week6_video_prez.pdf	Memory sharing/views; strides covered only as excluded-scope context.
Extras	Python Fundamentals; Classes, Iterators and Generators; Describing a Table; Tensors, Gradients and Regression; practice week 7 slides	Separate supplementary appendix, not merged into official TT1 scope.

Precision notes: where blind memorization can fail

- **Loss convention:** the lecture least-squares objective includes 1/(2m); generic “MSE” and torch.nn.MSELoss typically use 1/m. Both select the same minimizer without other changes, but the losses/gradients are scaled differently.
- **Regularized inverse:** L4 p. 23 presents $(X^TX+\lambda I)^{-1}X^Ty$; L5 p. 9 illustrates L2 penalizing $\theta_1 \ldots \theta_n$ but not θ_0. The exact matrix and λ coefficient must follow the chosen penalty. The guide retains both readings rather than silently claiming they are identical.
- **Global optima:** convexity establishes no bad local minima; the fine print includes rank/identifiability, step sizes, existence of finite minimizers, and the difference between linear/logistic models versus nonlinear MLPs.
- **View rules:** “reshape returns a view” is true in the explicit TT1 contiguous examples but not for every arbitrary ndarray. The guide uses exact cases instead of unsafe universal statements.
- **Test boundary:** official TT1 asks theoretical slides L1–L6 and practice NumPy P3–P6, not stride tricks. Later practice tools are separated as supplementary reading.